



Sergio de la Coba García

2ºASIR

ÍNDICE:

- 1. Introducción e Instalación del Entorno**
- 2. Desarrollo de la Aplicación Flask**
- 3. Configuración de Contenedores y Dockerización**
- 4. Despliegue de Recursos en Kubernetes**
- 5. Configuración del Servicio de Balanceo de Carga**
- 6. Pruebas de Acceso Local y Port-Forwarding**
- 7. Configuración de Seguridad en la Nube (AWS)**
- 8. Validación de Escalado y Conclusiones Finales**

Empecé instalando el motor del clúster. Utilicé el script oficial de K3s con curl configurando el modo de permisos en 644 para poder gestionar el archivo de configuración sin problemas. Inmediatamente después, descargué el binario de kubectl directamente desde los servidores de Google para tener la herramienta de administración actualizada en mi terminal de WSL.

```
alumno18@A6Alumno18:/mnt/c/Windows/System32$ curl -sL https://get.k3s.io | K3S_KUBECONFIG_MODE="644" sh -
[sudo] password for alumno18:
Sorry, try again.
[sudo] password for alumno18:
[INFO] Finding release for channel stable
[INFO] Using v1.34.3+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/k3s
```

```
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ curl -LO "
https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/st
able.txt)/bin/linux/amd64/kubectl"
  % Total    % Received % Xferd  Average Speed   Time    Time     T
ime Current              Dload  Upload  Total  Spent    L
eft  Speed
100 138 100 138 0 0 781 0 --:--:-- --:--:-- --:
--:-- 784
100 55.8M 100 55.8M 0 0 51.5M 0 0:00:01 0:00:01 --:
--:-- 68.2M
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo chmod
+x kubectl
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo mv ku
bectl /usr/local/bin/
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl ve
rsion --client
Client Version: v1.35.0
Kustomize Version: v5.7.1
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ mkdir -p $
HOME/.kube
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo cp /e
tc/rancher/k3s/k3s.yaml $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
sudo chmod 600 $HOME/.kube/config
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl ge
t nodes
NAME STATUS ROLES AGE VERSION
a6alumno18 Ready control-plane 32m v1.34.3+k3s1
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ mkdir -p ~
/kubernetes-aws-practice
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ cd ~/kuber
netes-aws-practice
alumno18@A6Alumno18:~/kubernetes-aws-practice$ |
```

PARTE 1: CREAR APLICACIÓN SIMPLE PARA KUBERNETES 1.1 – Crear carpeta para la aplicación

```
alumno18@A6Alumno18:~/kubernetes-aws-practice$ mkdir -p ~/kubernetes-aws-practice/app
alumno18@A6Alumno18:~/kubernetes-aws-practice$ cd ~/kubernetes-aws-practice/app
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ INFO[0608] COMPACT compactRev=0 targetCompactRev=255 currentRev=1255
INFO[0608] COMPACT deleted 33 rows from 255 revisions in 14.248827ms - compacted to 255/1255
INFO[0608] COMPACT compacted from 0 to 255 in 1 transactions over 15ms
I0204 13:04:42.372561 8075 cidrallocator.go:277] updated Cluster IP allocator for Service CIDR 10.43.0.0/16
```

1.2 – Crear aplicación Python con Flask

Aquí comencé con el desarrollo de la aplicación. Creé el archivo app.py utilizando Flask. Como puede ver, definí la ruta principal / para renderizar el index, una ruta /pod-info para extraer el nombre del pod mediante la librería os (fundamental para demostrar el balanceo más adelante) y una ruta /health para los chequeos de salud

```
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ cat > app.py <<
'EOF'
#!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys
app = Flask(__name__)
# Variables de entorno inyectadas por Kubernetes
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')
@app.route('/')
def index():
    return send_from_directory('.', 'index.html')
@app.route('/pod-info')
def pod_info():
    return jsonify({
        'pod_name': POD_NAME,
        'namespace': POD_NAMESPACE,
        'hostname': socket.gethostname(),
        'timestamp': datetime.now().isoformat()
    })
@app.route('/health')
def health():
    return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200
if __name__ == '__main__':
    print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
    app.run(host='0.0.0.0', port=5000, debug=False)
EOF
```

```

alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ chmod +x app.py
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ tail -n 3 app.py
if __name__ == '__main__':
    print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.std
err)
    app.run(host='0.0.0.0', port=5000, debug=False)
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ INFO[0908] COMPA
CT compactRev=255 targetCompactRev=345 currentRev=1345
INFO[0908] COMPACT deleted 13 rows from 90 revisions in 724.937µs -
compacted to 345/1345
INFO[0908] COMPACT compacted from 255 to 345 in 1 transactions over
1ms

```

CREAR EL ARCHIVO HTML:

Diseñé un archivo index.html sencillo pero funcional. Lo importante aquí es el uso de variables de Jinja2 ({{ pod_name }}) para que la interfaz muestre visualmente en el navegador exactamente qué Pod está respondiendo a la solicitud

```

alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ cat > index.html
<< 'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>Kubernetes Load Balancer</title>
    <style>
        body { font-family: Arial, sans-serif; text-align: center;
margin-top: 50px; }
        .info { background: #f4f4f4; padding: 20px; border-radius:
10px; display: inline-block; }
    </style>
</head>
<body>
    <h1>Kubernetes Load Balancer</h1>
    <div class="info">
        <p>Pod atendiendo solicitud:</p>
        <h2 id="pod-name">Cargando...</h2>
    </div>
    <p>Actualiza constantemente esta página para ver el balanceo de
carga en acción</p>
    <button onclick="location.reload()">Actualizar Ahora</button>
</body>
</html>
EOF

```

1.4 – Crear requirements.txt

Este archivo le dice a Docker (o a tu sistema) exactamente qué versiones de Flask necesita instalar.

```
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ cat > requirements.txt << 'EOF'
Flask==3.0.0
Werkzeug==3.0.0
EOF
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ |
```

1.5 – Crear el Dockerfile

Este es el "manual de instrucciones" para crear la imagen de tu contenedor.

Para empaquetar la aplicación, configuré el Dockerfile. Usé una imagen base ligera de Python 3.9, definí el directorio de trabajo en /app, copié los archivos necesarios e instalé las dependencias. Expuse el puerto 5000, que es donde corre Flask por defecto

```
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
# Copiar archivos
COPY requirements.txt .
COPY app.py .
COPY index.html .
# Instalar dependencias
RUN pip install --no-cache-dir -r requirements.txt
# Ejecutar app
CMD ["python", "app.py"]
EOF
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ |
```

verificar:

```
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ ls -F
Dockerfile  app.py*  index.html  requirements.txt
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ |
```


2.1 – Crear Namespace

Primero preparamos el "contenedor lógico" donde vivirá todo nuestro proyecto.

Para mantener las buenas prácticas y el orden en el clúster, lo primero que hice fue crear un Namespace

Así separo los recursos de esta práctica de cualquier otro proceso del sistema.

```
alumno18@A6Alumno18:~/kubernetes-aws-practice/app$ cd ~/kubernetes-aws-practice

cat > namespace.yaml << 'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: load-balancer-demo
  labels:
    name: load-balancer-demo
EOF

# Aplicar y verificar
kubectl apply -f namespace.yaml
kubectl get namespaces
I0204 13:29:33.968144 8075 controller.go:667] quota admission ad
ded evaluator for: namespaces
namespace/load-balancer-demo created
NAME                STATUS    AGE
default              Active    59m
kube-node-lease      Active    59m
kube-public          Active    59m
kube-system          Active    59m
load-balancer-demo   Active    1s
alumno18@A6Alumno18:~/kubernetes-aws-practice$
```

2.2 – Crear ConfigMap

Este paso es clave: estamos guardando el código de tu app dentro de Kubernetes para que los Pods puedan leerlo sin necesidad de construir una imagen personalizada todavía.

En lugar de reconstruir la imagen para cada cambio, utilicé un ConfigMap. Aquí pegué el contenido de app.py e index.html. Esto permite que Kubernetes 'monte' estos archivos dentro de los contenedores de forma dinámica, facilitando la gestión del código.

```
alumno18@A6Alumno18:~/kubernetes-aws-practice$ cat > configmap.yaml
<< 'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-files
  namespace: load-balancer-demo
data:
  requirements.txt: |
    Flask==3.0.0
    Werkzeug==3.0.0
  app.py: |
    #!/usr/bin/env python3
    from flask import Flask, jsonify, send_from_directory
    import os
    import socket
    from datetime import datetime
    import sys
    app = Flask(__name__)
    POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
    POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')
    @app.route('/')
    def index():
        return send_from_directory('.', 'index.html')
    @app.route('/pod-info')
    def pod_info():
        return jsonify({
            'pod_name': POD_NAME,
            'namespace': POD_NAMESPACE,
            'hostname': socket.gethostname(),
            'timestamp': datetime.now().isoformat()
        })
    @app.route('/health')
    def health():
        return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200

    if __name__ == '__main__':
        print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys
.stderr)
        app.run(host='0.0.0.0', port=5000, debug=False)
  index.html: |
    <!DOCTYPE html>
    <html>
    <head><title>Kubernetes Load Balancer</title></head>
    <body>
      <h1>Kubernetes Load Balancer</h1>
      <p>Pod atendiendo solicitud: <b id="pod-name">Cargando...</b>
    </p>
      <button onclick="location.reload()">Actualizar Ahora</button>
    </body>
```



```

n>
    </body>
kubectl get configmap -n load-balancer-demo
configmap/app-files created
NAME          DATA   AGE
app-files     3       0s
kube-root-ca.crt 1       37s

```

2.3 – Crear Deployment (Las 3 réplicas)

Configuré el archivo deployment.yaml. Definí 3 réplicas para asegurar la disponibilidad. En la sección de volumes, vinculé el ConfigMap creado anteriormente a las rutas /app/app.py y /app/templates/index.html dentro del contenedor. Esto es lo que permite que mis Pods ejecuten el código que definí."

```

alumno18@A6Alumno18:~/kubernetes-aws-practice$ cat > deployment.yaml
l << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web-app
        image: python:3.11-slim
        command: ["sh", "-c"]
        args:
        - |
          cd /app
          pip install --no-cache-dir -r requirements.txt > /dev/nul
l 2>&l
          python app.py
        ports:
        - containerPort: 5000
          protocol: TCP
        env:
        - name: POD_NAME
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
        - name: POD_NAMESPACE
          valueFrom:
            fieldRef:
              fieldPath: metadata.namespace
        volumeMounts:
        - name: app-volume
          mountPath: /app
        livenessProbe:
          httpGet:
            path: /health
            port: 5000
          initialDelaySeconds: 15

```

NAME	READY	STATUS	RESTARTS	AGE
web-app-65967466dd-8w7lv	1/1	Running	0	19s
web-app-65967466dd-d8r6f	1/1	Running	0	19s
web-app-65967466dd-v4bks	1/1	Running	0	19s

2.4 – Crear Service (Load Balancer)

Este comando creará la puerta de entrada para que puedas ver tu app desde el navegador.

```
alumno18@A6Alumno18:~/kubernetes-aws-practice$ cat > service.yaml <
< 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: web-app-service
  namespace: load-balancer-demo
  labels:
    app: web-app
spec:
  type: LoadBalancer
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
      name: http
  sessionAffinity: None
EOF

# Aplicar y obtener la IP
kubectl apply -f service.yaml
kubectl get svc -n load-balancer-demo web-app-service
I0204 13:32:30.734671 8075 alloc.go:328] "allocated clusterIPs"
service="load-balancer-demo/web-app-service" clusterIPs={"IPv4":"10
.43.198.130"}
I0204 13:32:30.735264 8075 event.go:389] "Event occurred" object
="load-balancer-demo/web-app-service" fieldPath="" kind="Service" a
piVersion="v1" type="Normal" reason="EnsuringLoadBalancer" message=
"Ensuring load balancer"
service/web-app-service created
I0204 13:32:30.747648 8075 controller.go:667] quota admission ad
ded evaluator for: daemonsets.apps
I0204 13:32:30.750593 8075 event.go:389] "Event occurred" object
="load-balancer-demo/web-app-service" fieldPath="" kind="Service" a
piVersion="v1" type="Normal" reason="AppliedDaemonSet" message="App
lied LoadBalancer DaemonSet kube-system/svc-lb-web-app-service-e3bcf
41f"
I0204 13:32:30.757925 8075 controller.go:667] quota admission ad
ded evaluator for: controllerrevisions.apps
NAME              TYPE              CLUSTER-IP      EXTERNAL-IP      PORT
(S)              AGE
web-app-service   LoadBalancer     10.43.198.130    <pending>        80:3
1943/TCP          0s
alumno18@A6Alumno18:~/kubernetes-aws-practice$ |
```


CONFIGURACIÓN EN AWS:

Como el clúster corre en una instancia de EC2, tuve que configurar las reglas de entrada. Abrí el puerto 22 para SSH, el 80 para el tráfico HTTP estándar y el 443 para HTTPS, asegurando que el origen sea 0.0.0.0/0 para que sea accesible desde cualquier sitio

Reglas de entrada.

Reglas de entrada [Información](#)

Regla de entrada 1 [Eliminar](#)

Tipo Información	Protocolo Información
SSH ▼	TCP
Intervalo de puertos Información	Tipo de origen Información
22	Anywhere-IPv4 ▼
Origen Información	Descripción: opcional Información
Q 0.0.0.0/0	
0.0.0.0/0 ✕	

Regla de entrada 2 [Eliminar](#)

Tipo Información	Protocolo Información
HTTP ▼	TCP
Intervalo de puertos Información	Tipo de origen Información
80	Anywhere-IPv4 ▼
Origen Información	Descripción: opcional Información
Q 0.0.0.0/0	
0.0.0.0/0 ✕	

Regla de entrada 3 [Eliminar](#)

Tipo Información	Protocolo Información
HTTPS ▼	TCP
Intervalo de puertos Información	Tipo de origen Información
443	Anywhere-IPv4 ▼
Origen Información	Descripción: opcional Información
Q 0.0.0.0/0	
0.0.0.0/0 ✕	

[Agregar regla](#)

▼ Reglas de entrada

Q Filtrar reglas < 1 >

Nombre	ID de la regla del grupo ...	Intervalo de p...	Protocolo	Origen	Grupos de seguridad	Descripción
-	sgr-05fe747912eebb962	22	TCP	0.0.0.0/0	kubernetes-aws-sg 🔗	-
-	sgr-0c612eec5fbdcd8d63	443	TCP	0.0.0.0/0	kubernetes-aws-sg 🔗	-
-	sgr-0b1137b82f97bc833	80	TCP	0.0.0.0/0	kubernetes-aws-sg 🔗	-

Nos conectamos a la instancia desde wsl:

Me conecté por SSH a la instancia. Aquí realicé un `sudo apt update` para sincronizar los repositorios y creé el directorio `kubernetes` para clonar o mover allí mis archivos YAML, asegurando que todo el despliegue se pueda replicar desde la consola de la instancia

```
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ ssh -i ~/.ssh/wordpress-key-aws.pem ubuntu@18.215.15.202
The authenticity of host '18.215.15.202 (18.215.15.202)' can't be established.
ED25519 key fingerprint is SHA256:FmZhyy0kvvRRmMnx8ijaMtDQ8CLoOKWSGFWF8s43gw0.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:2: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.215.15.202' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Wed Feb  4 13:08:26 UTC 2026

System load:  0.01          Temperature:   -273.1 C
Usage of /:   42.9% of 6.71GB Processes:    122
Memory usage: 68%          Users logged in: 1
Swap usage:   0%           IPv4 address for ens5: 172.31.14.67

* Ubuntu Pro delivers the most comprehensive open source security and compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

22 updates can be applied immediately.
13 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Feb  4 13:06:21 2026 from 18.206.107.27
ubuntu@ip-172-31-14-67:~$
```

Actualizamos los paquetes y creamos la carpeta para kubernetes.

```
ubuntu@ip-172-31-14-67:~$ mkdir -p ~/kubernetes-test
cd ~/kubernetes-test
ubuntu@ip-172-31-14-67:~/kubernetes-test$
```

Creamos el script de prueba:

Para demostrar el balanceo, programé un pequeño script en Bash que hace peticiones en bucle. Como se ve en la captura, al ejecutarlo, las respuestas muestran nombres de Pods diferentes en cada línea. Esto confirma que el balanceador está repartiendo la carga proporcionalmente entre las 3 réplicas."

```
ubuntu@ip-172-31-14-67:~/kubernetes-test$ cat > ~/test-kubernetes-lb.sh
b.sh << 'EOF'
#!/bin/bash
echo "=== Prueba Kubernetes Load Balancer desde AWS ==="
echo ""
declare -A pods_count
for i in {1..15}; do
    response=$(curl -s http://localhost:8888/pod-info)
    pod_name=$(echo $response | python3 -c "import sys, json; print(json.load(sys.stdin)['pod_name'])" 2>/dev/null)
    if [ -z "$pod_name" ]; then
        pod_name="ERROR"
    fi
    echo "Petición $i → Pod: $pod_name"
    pods_count[$pod_name]=$(( ${pods_count[$pod_name]:-0} + 1 ))
    sleep 1
done
echo ""
echo "=== Resumen Balanceo ==="
for pod in "${!pods_count[@]}"; do
    echo "Pod $pod: ${pods_count[$pod]} peticiones"
done
EOF

sudo chmod +x ~/test-kubernetes-lb.sh
> EOF
ubuntu@ip-172-31-14-67:~/kubernetes-test$
```



```

ubuntu@ip-172-31-14-67:~$ ./test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 → Pod: web-app-65967466dd-v4bks
Petición 2 → Pod: web-app-65967466dd-v4bks
Petición 3 → Pod: web-app-65967466dd-v4bks
Petición 4 → Pod: web-app-65967466dd-v4bks
Petición 5 → Pod: web-app-65967466dd-v4bks
Petición 6 → Pod: web-app-65967466dd-v4bks
Petición 7 → Pod: web-app-65967466dd-v4bks
Petición 8 → Pod: web-app-65967466dd-v4bks
Petición 9 → Pod: web-app-65967466dd-v4bks
Petición 10 → Pod: web-app-65967466dd-v4bks
Petición 11 → Pod: web-app-65967466dd-v4bks
Petición 12 → Pod: web-app-65967466dd-v4bks
Petición 13 → Pod: web-app-65967466dd-v4bks
Petición 14 → Pod: web-app-65967466dd-v4bks
Petición 15 → Pod: web-app-65967466dd-v4bks

=== Resumen Balanceo ===
Pod web-app-65967466dd-v4bks: 15 peticiones
ubuntu@ip-172-31-14-67:~$ |

```

Lanzamos escalado en la máquina local con wsl

```

alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ # Aumentar
a 5 replicas
kubectl scale deployment web-app -n load-balancer-demo --replicas=5

# Verificar que están naciendo
kubectl get pods -n load-balancer-demo

# Esperar a que los 5 estén listos
kubectl wait --for=condition=ready pod -l app=web-app -n load-balancer-demo --timeout=120s
deployment.apps/web-app scaled

```

NAME	READY	STATUS	RESTARTS
web-app-65967466dd-27tr9	0/1	ContainerCreating	0
web-app-65967466dd-8w7lv	1/1	Running	1 (24m ago)
web-app-65967466dd-9f6vt	0/1	ContainerCreating	0
web-app-65967466dd-d8r6f	1/1	Running	1 (24m ago)
web-app-65967466dd-v4bks	1/1	Running	1 (24m ago)

```

pod/web-app-65967466dd-27tr9 condition met
pod/web-app-65967466dd-8w7lv condition met
pod/web-app-65967466dd-9f6vt condition met
pod/web-app-65967466dd-d8r6f condition met
pod/web-app-65967466dd-v4bks condition met

```

Y volvemos a lanzar el balanceo en la instancia

```

ubuntu@ip-172-31-14-67:~$ ~/test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 → Pod: web-app-65967466dd-v4bks
Petición 2 → Pod: web-app-65967466dd-v4bks
Petición 3 → Pod: web-app-65967466dd-v4bks
Petición 4 → Pod: web-app-65967466dd-v4bks
Petición 5 → Pod: web-app-65967466dd-v4bks
Petición 6 → Pod: web-app-65967466dd-v4bks
Petición 7 → Pod: web-app-65967466dd-v4bks
Petición 8 → Pod: web-app-65967466dd-v4bks
Petición 9 → Pod: web-app-65967466dd-v4bks
Petición 10 → Pod: web-app-65967466dd-v4bks
Petición 11 → Pod: web-app-65967466dd-v4bks
Petición 12 → Pod: web-app-65967466dd-v4bks
Petición 13 → Pod: web-app-65967466dd-v4bks
Petición 14 → Pod: web-app-65967466dd-v4bks
Petición 15 → Pod: web-app-65967466dd-v4bks

=== Resumen Balanceo ===
Pod web-app-65967466dd-v4bks: 15 peticiones

```

MONITOREO Y OBSERVABILIDAD:

Finalmente, dejé la terminal monitorizando los logs y el estado de los recursos para comprobar que, bajo carga, los contenedores se mantenían estables y no había errores de conexión

LOGS EN TIEMPO REAL.

```

alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl log
s -n load-balancer-demo -l app=web-app -f
10.42.0.1 - - [04/Feb/2026 13:56:26] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:26] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:28] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:33] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:36] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:38] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:43] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:46] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:48] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:53] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:56] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:26] "GET /health HTTP/1.1" 200 -
10.42.0.1 - - [04/Feb/2026 13:56:21] "GET /health HTTP/1.1" 200 -

```

Las métricas de rendimiento permiten monitorizar el consumo de CPU (procesamiento) y RAM (memoria) de los Pods para ajustar el número de réplicas y evitar caídas o desperdicio de recursos en AWS. Por su parte, el historial de cambios ofrece trazabilidad sobre las actualizaciones del despliegue, funcionando como un "seguro de vida" que permite realizar rollbacks inmediatos a versiones estables si algo falla. En conjunto, estas herramientas demuestran una gestión profesional del clúster, garantizando que la infraestructura sea eficiente, escalable y capaz de recuperarse ante errores de configuración de forma rápida.

Métricas de rendimiento:

```
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl top
pods -n load-balancer-demo
NAME                                CPU(cores)   MEMORY(bytes)
web-app-65967466dd-27tr9            2m           24Mi
web-app-65967466dd-8w7lv            2m           28Mi
web-app-65967466dd-9f6vt            2m           24Mi
web-app-65967466dd-d8r6f            2m           29Mi
web-app-65967466dd-v4bks            2m           29Mi
```

Historial de cambios.

```
alumno18@A6Alumno18:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl rollout history deployment web-app -n load-balancer-demo
deployment.apps/web-app
REVISION  CHANGE-CAUSE
1          <none>
```